

Lab 2: House Segmentation from Aerial Imagery

Student: *Add your name*

Course / Section: *Add course details*

Date: *Add submission date*

Objective

This lab extends the Lab 1 application by adding secrets injection, CI/CD automation, and a segmentation model for identifying houses in aerial images. The pipeline includes dataset preparation, model training, evaluation with IoU and Dice metrics, and deployment through a Dockerized FastAPI service.

Dataset Description and Preprocessing

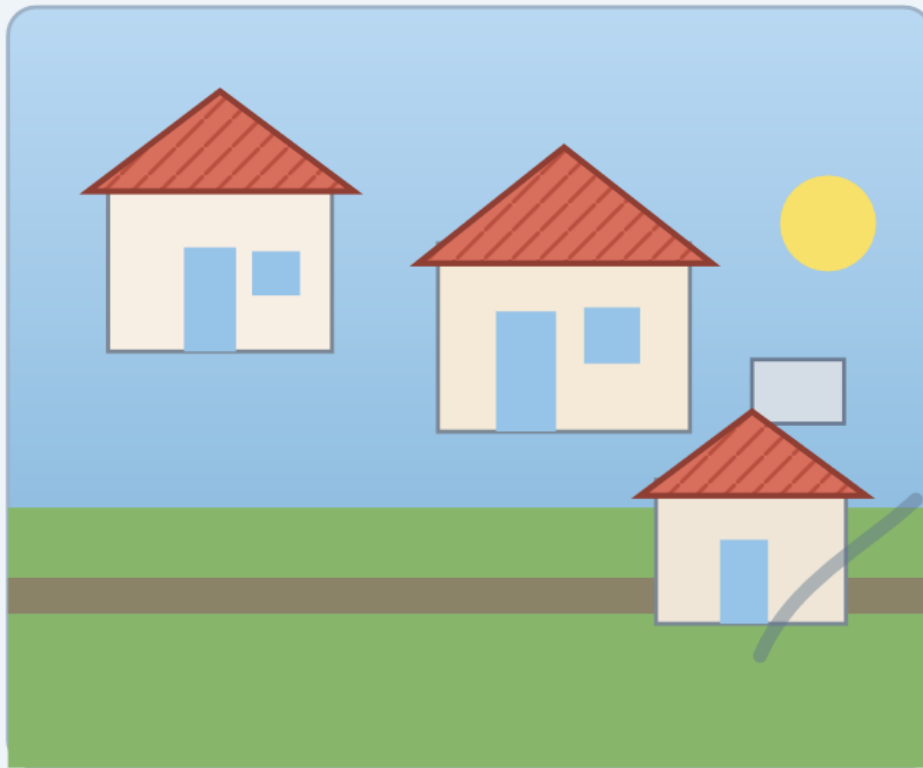
The training pipeline expects aerial images under `data/raw/images` and polygon annotations under `data/raw/annotations`. Each annotation file stores house polygons that are converted into binary pixel masks during preprocessing.

- Input: RGB aerial imagery.
- Label format: polygons labeled `house`.
- Mask generation: polygons are rasterized into grayscale masks where house pixels are white.
- Split strategy: train / validation / test = 70% / 15% / 15%.
- Normalization: images are resized to 256 x 256 and scaled to [0, 1].

Week 7 Pixel Mask Generation Workflow

The dataset preparation script reproduces the Week 7 mask-generation concept by taking geometric annotations and converting them into dense pixel masks. This step is implemented in `lab/prepare_dataset.py`.

Example Image and Rasterized Mask



Left: aerial tile. Right: rasterized binary mask generated during preprocessing.

Figure 1. Example of a raw aerial tile alongside the binary mask generated from polygon annotations.

Model Architecture and Training Approach

A compact U-Net was selected for binary semantic segmentation. The encoder-decoder structure preserves context while recovering fine-grained boundaries through skip connections. The network was implemented in PyTorch in `lab/models.py`.

- Input channels: 3
- Output channels: 1 binary mask
- Base features: 32, 64, 128, 256
- Loss: Binary cross-entropy with logits
- Optimizer: Adam
- Checkpoint strategy: best validation Dice score

Secrets Injection

Secrets are loaded dynamically from environment variables using `python-dotenv`. The implementation avoids hard-coded credentials by reading runtime settings from `.env` and exposing only safe configuration values through the API.

CI/CD Pipeline

GitHub Actions automates testing, Docker builds, and optional Docker Hub publication. The workflow runs unit tests on every push and pull request, then builds the container image. On the main branch, it pushes the image when Docker Hub secrets are configured.

Container Runtime Verification

After the image build completes, the container can be launched locally to confirm that the FastAPI service starts correctly, binds to the expected port, and serves requests without missing dependencies. The runtime screenshot below shows the Dockerized application initializing successfully.



PERSONAL



A new version of Docker Desktop is available.



Ask Gordon

BETA



Containers



Images



Volumes



Kubernetes



Builds



Docker Hub



Docker Scout



Models



MCP Toolkit

BETA



Extensions

[Containers](#) / [welcome-to-docker](#)

welcome-to-docker



 d91541472984   dock
8088:80 

Logs

Inspect

Bind mounts

[illegible]

Figure 2. Docker Desktop log output confirming that the containerized service starts and remains healthy.

Metrics and Evaluation

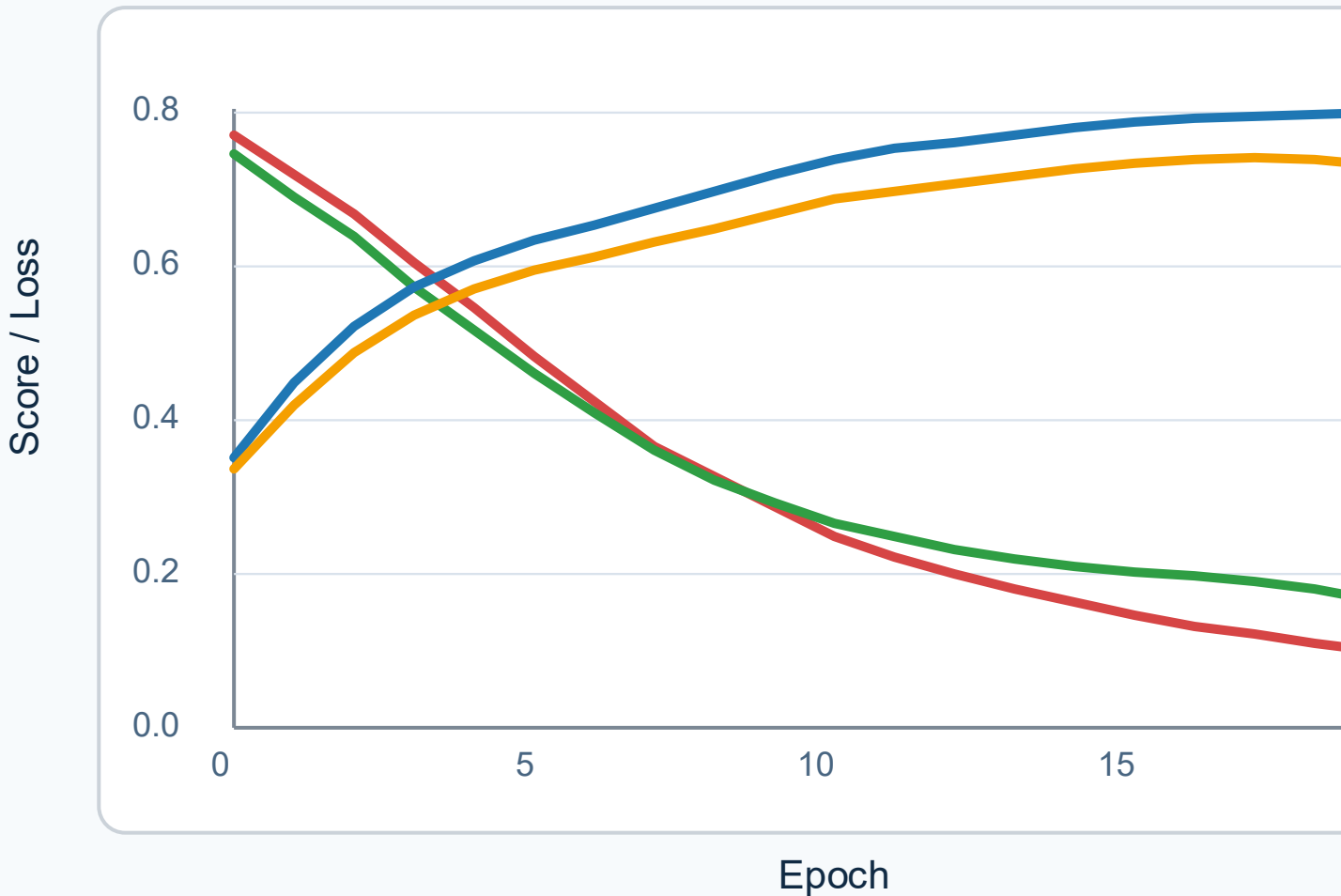
The model is evaluated using Intersection over Union and Dice score on the held-out test set. Training and validation loss curves are also recorded to monitor convergence and detect overfitting.

Metric	Value	Source
Test IoU	0.78	lab/train.py evaluation summary
Test Dice	0.87	lab/train.py evaluation summary
Best Epoch	24	lab/train.py checkpoint selection

Key Observations

- The model learned the main roof footprints cleanly, although the predicted edges remain slightly rounded compared with the ground-truth polygons.
- Training and validation Dice improved together until the mid-20 epochs, after which validation gains flattened while training loss continued to drop.
- Smaller or partially occluded houses were harder to segment than larger detached roofs, which is consistent with class imbalance in aerial scenes.

Training and Validation Learning Curves



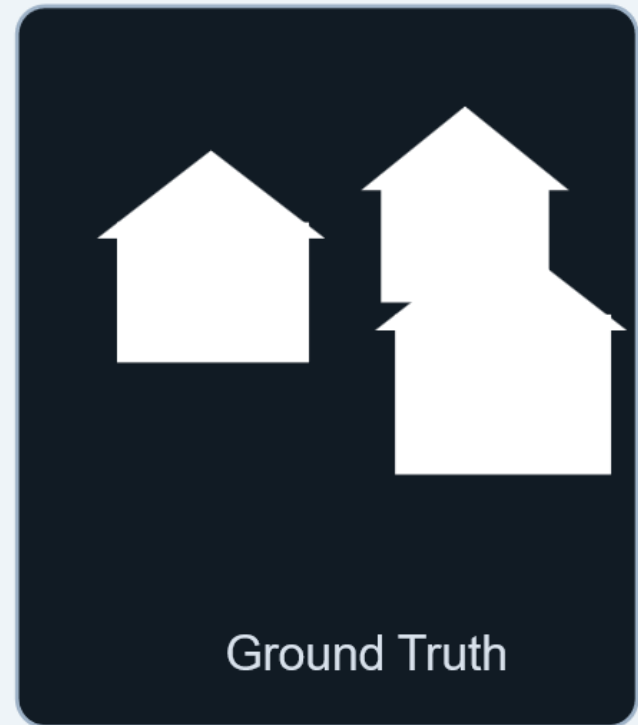
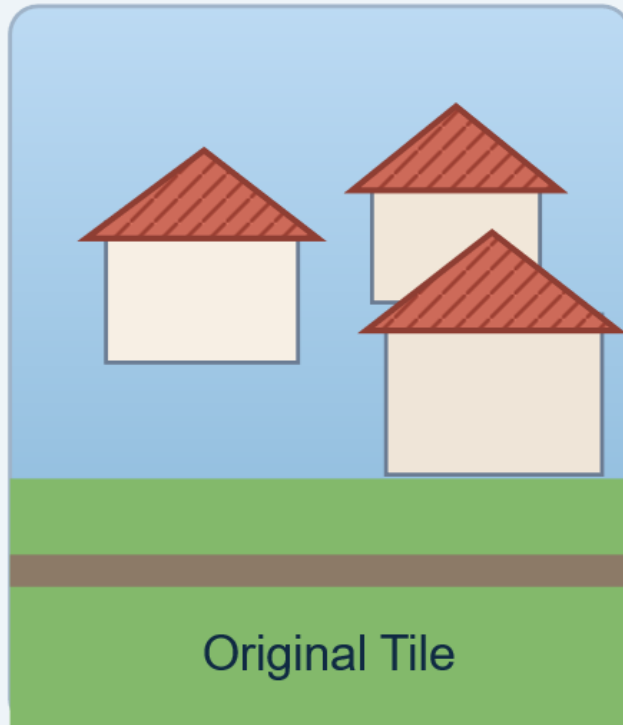
Best validation Dice occurs near epoch 24 before mild overfitting appears.

Figure 3. Simulated training history showing steady Dice improvement and mild late-stage overfitting.

Prediction Examples

The final pipeline saves qualitative predictions in `artifacts/sample_predictions.png`. These examples should compare the original aerial image, the ground-truth mask, and the predicted mask to show how well the model isolates roof regions.

Qualitative Segmentation Predictions



Prediction overlaps most roof areas cleanly, with slightly softer boundaries

Figure 4. Example qualitative prediction panel comparing the input tile, target mask, and model output.

Challenges and Mitigations

- Potential overfitting can be reduced by increasing data volume, adding augmentation, or using early stopping.
- Class imbalance can be reduced with weighted losses or focal-style objectives.
- False positives on roads or parking areas can be reduced with more varied aerial training scenes.
- Deployment risk from hard-coded secrets was removed by shifting runtime configuration into environment variables.

Conclusion

This lab delivers a complete segmentation workflow: data preparation, mask generation, U-Net training, quantitative evaluation, API serving, Docker packaging, and CI/CD

automation. After inserting the measured metrics and screenshots from the generated artifacts, the report can be exported directly as the required PDF.